BIOINFORMATICS

Workshop 5: HP-Lattice Folding

Prof. Magnus Bordewich

Learning Objectives

- Implement brute-force enumeration of HP-lattice conformations.
- Implement a simple heuristic search for longer chains where brute force becomes infeasible.
- Reflect on what the results imply about the NP-hardness of structure prediction.

Background

The HP lattice model reduces protein folding to: a two-letter (H/P) sequence, conformations restricted to self-avoiding walks on a 2D or 3D lattice, and an energy of -1 for every lattice-adjacent, chain-non-consecutive pair of H residues. Even this drastic simplification is NP-hard to solve exactly, and the number of self-avoiding walks grows exponentially with chain length – so heuristics are required.

Coding Activity

Given an input sequence s, a string consisting of Hs and Ps, we assume s[0] is located at [0,0] and s[1] at [1,0]. We can then represent the 2D lattice structure as a list of Forward, Left, Right decisions indicating if the next residue s[i] continues the direction of s[i-1]->s[i-2] or makes a left or right turn.

1) Write generate_coords(turns) which generates a list of coordinates, starting with [0,0], [1,0], corresponding to the coordinates of residues given the turns.
2) Write is_valid_walk(coords) checking a list of lattice coordinates forms a self-avoiding walk (no repeated points, each step to a lattice neighbour of the last).
3) Write energy(sequence, coords) computing the HP contact energy: -1 for every pair of residues i<j with sequence[i]==sequence[j]=='H', |i-j|>1, and coords[i], coords[j] lattice-adjacent.
4) Write enumerate_conformations(sequence) that brute-force enumerates every self-avoiding walk realising sequence on the 2D square lattice (fixing the first step's direction to avoid trivially counting rotations/reflections separately), and returns the best (minimum) energy found, the conformations achieving it, and the total number examined.
5) Reproduce the lecture's table: run your function on prefixes of length 6, 8, 10, 12 (and 14, if your machine is fast enough) of HPHPPHHPHHPPHPHH, and confirm the counts and best energies match the lecture slide.
6) Write a simple greedy heuristic search – sequentially extending a turns list with a random minimum energy non-intersecting turn. If the chain gets trapped into an intersection, restart. Run a large number of times and keep the best result. How often do you need to restart?

7) Write a simulated-annealing move-based search over self-avoiding walks - that tries to find a low-energy conformation for a chain too long to brute-force (length 20-30).

   a) Structure is the turns list: start with turns= FFFFF…F (all Forwards) and T=2.0

   b) Select a turn, i, randomly, and a direction d in {F,L,R} randomly, set turns'[i]=d and turns'[j]=turns[j], j!=i

   c) Accept if the energy of turns' is lower than that of turns, or with probability exp((energy(turns)-energy(turns'))/T) otherwise, else stay at turns.

   d) Decrease the temperature T (T=T*a for some constant a, e.g. 0.9995)

   e) Repeat b)-d) for many iterations or until the energy has not decreased for 100 iterations, say.

8) Compare its best-found energy against the true brute-force optimum and against the greedy approach. Try tuning the temperature decay a.

9) Test data: these are classic sequences from Unger & Moult with their known optimal 2D energies. You should be able to find them in ~100000 iterations

Length	Sequence	Optimal E
20	HPHPPHHPHPPHPHHPPHPH	−9
24	HHPPHPPHPPHPPHPPHPPHPPHH	−9
25	PPHPPHHPPPPHHPPPPHHPPPPHH	−8

Unger, R. & Moult, J. (1993). Genetic algorithms for protein folding simulations. *Journal of Molecular Biology*, 231(1), 75–81. https://doi.org/10.1006/jmbi.1993.1258

Extension Tasks (Optional)

a) Extend your model to the 3D cubic lattice.

b) Generate an animation of the simulated annealing.

c) Extend the set of moves considered in simulated annealing, e.g. to include end moves, corner flips, crankshaft moves or pull moves, which keep the chain valid. (see e.g. Lesh, N., Mitzenmacher, M. & Whitesides, S. (2003). A complete and effective move set for simplified protein folding. In: Proc. RECOMB 2003, pp. 188–195. Available as a MERL technical report (TR2003-03): https://www.merl.com/publications/docs/TR2003-03.pdf)

d) Try a different approach altogether: e.g. genetic algorithms. Unger, R. & Moult, J. (1993). Genetic algorithms for protein folding simulations. *Journal of Molecular Biology*, 231(1), 75–81. https://doi.org/10.1006/jmbi.1993.1258